

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра математического обеспечения и применения ЭВМ

ОТЧЕТ
по лабораторной работе №2
по дисциплине «Построение и анализ алгоритмов»
Тема: Задача Коммивояжёра

Студент гр. 4345

Кравцов С. А.

Преподаватель

Жангиров Т.Р.

Санкт-Петербург

2026

Цель работы

Изучить и реализовать алгоритмы решения задачи коммивояжёра.

Постановка задачи

Напишите программу, решающую задачу коммивояжёра. Нужно найти кратчайший маршрут, который проходит через все заданные города ровно один раз и возвращается в исходный город. Не все города могут быть напрямую связаны друг с другом.

Входные данные:

n — количество городов ($5 \leq n \leq 15$).

Матрица расстояний между городами размером $n \times n$, где $graph[i][j]$ обозначает расстояние от города i до города j . Если $graph[i][j]=0$ (и $i \neq j$), это означает, что прямого пути между городами нет.

Выходные данные:

Минимальная стоимость маршрута, проходящего через все города и возвращающегося в начальный город.

Оптимальный путь в виде последовательности посещаемых городов, начинающейся и заканчивающейся в городе 0.

Если такого пути не существует, вывести "no path".

Выполнение работы

При полном переборе задачи коммивояжёра очевидным кажется решение, имеющее факториальную сложность $O(n!)$, где n — количество городов. Такая сложность обусловлена тем, что необходимо проверить все возможные перестановки маршрутов, проходящих через каждый город ровно один раз. При увеличении числа вершин количество вариантов возрастает крайне быстро, что делает полный перебор практически неприменимым уже при сравнительно небольших размерах входных данных. Для уменьшения количества повторных вычислений применяется динамическое программирование. Алгоритм Хелда–Карпа использует разбиение задачи на подзадачи, каждая из которых описывается множеством уже посещённых городов и текущей вершиной маршрута. Благодаря сохранению результатов вычисленных состояний исключается повторный расчёт одинаковых фрагментов пути, что позволяет сократить сложность до $O(2^n * n^2)$.

Были реализованы методы `tsp_dp` и `alsh_1`. Первый представляет собой точный алгоритм динамического программирования Хелда–Карпа, второй — приближённый жадный алгоритм лучшего шага.

Алгоритм Хелда–Карпа — процесс начинается из стартового города 0, который считается уже посещённым. Основной рекурсивный метод `dp(mask, u)` вычисляет минимальную стоимость маршрута, если множество городов `mask` уже посещено, а текущая позиция находится в вершине `u`. Битовая маска используется для компактного хранения информации о посещённых вершинах: каждый бит соответствует отдельному городу. Если все вершины уже посещены, алгоритм завершает маршрут возвратом в исходный город. В противном случае выполняется перебор всех ещё не посещённых вершин. Для каждой допустимой вершины вычисляется стоимость перехода и рекурсивно определяется минимальная стоимость оставшейся части пути. При нахождении более выгодного маршрута обновляется минимальное значение и сохраняется следующий город в массив `parent`, который затем используется для восстановления итогового пути. Массив `memo` реализует мемоизацию: если состояние `(mask, u)` уже вычислялось ранее, результат сразу возвращается без повторного обхода. Благодаря этому каждая подзадача рассчитывается

только один раз, что позволяет существенно уменьшить количество операций по сравнению с полным перебором.

Сложность алгоритма Хелда–Карпа по времени $O(2^N * N^2)$, где N – количество городов. Обусловлена тем, что алгоритм рассматривает все возможные подмножества посещённых вершин, количество которых составляет 2^N . Для каждого состояния дополнительно перебираются возможные текущие и следующие вершины, что даёт множитель N^2 . Благодаря мемоизации каждое состояние вычисляется только один раз, что значительно уменьшает количество повторных вычислений по сравнению с полным перебором.

Сложность алгоритма Хелда–Карпа по памяти $O(2^N * N)$, где N – количество городов. Обусловлена хранением таблиц `memo` и `parent`, содержащих значения для всех состояний динамического программирования, задаваемых комбинацией битовой маски и текущей вершины.

Жадный алгоритм лучшего шага (`alsh_1`) — процесс также начинается из вершины 0 и последовательно выбирает следующую вершину из числа непосещённых. На каждом шаге для всех допустимых переходов из текущего города вычисляется стоимость ребра и дополнительная оценка L . Функция `calc_L` оценивает потенциальную «выгодность» текущего состояния маршрута, учитывая минимальные возможные входы в стартовую вершину и выходы из конечной среди ещё не посещённых вершин. Эти значения усредняются и добавляются к локальной стоимости перехода. Далее выбирается вершина с минимальным значением $s + L$, она добавляется в маршрут и помечается как посещённая. Процесс повторяется, пока не будут посещены все вершины. В конце проверяется возможность возврата в начальную вершину 0. Если ребро существует, оно добавляется в маршрут и считается итоговая стоимость, иначе возвращается сообщение о невозможности построить путь. Алгоритм является жадным и работает быстрее точных методов, но не гарантирует оптимального решения.

Сложность данного жадного алгоритма по времени составляет примерно $O(N^3)$, где N — количество городов. Это обусловлено тем, что основной цикл

выполняется N раз, и на каждом шаге происходит перебор всех непосещённых вершин ($O(N)$). Дополнительно для каждой такой вершины вызывается функция `calc_L`, внутри которой также выполняется проход по всем вершинам графа ($O(N)$). В результате вложенность операций приводит к кубической сложности в худшем случае.

Сложность по памяти составляет $O(N)$, где N — количество городов. Это связано с хранением массива посещённых вершин `visited`, текущего маршрута `path` и вспомогательных переменных. Матрица графа не учитывается как дополнительная память алгоритма, так как она является входными данными.

Исходный код программы см. в приложении А.

Результаты тестирования см. в Таблице 1-2.

Таблица 1 – Тестирование `tsp_dp`

№	Входные данные	Выходные данные
1	5 0 3 2 6 13 9 0 10 9 11 6 16 0 5 6 6 6 11 0 9 10 14 3 5 0	28 0 1 4 2 3 0
2	5 0 8 1 18 5 8 0 15 4 12 1 15 0 4 1 18 4 4 0 10 5 12 1 10 0	22 0 1 3 2 4 0

Таблица 2 – Тестирование `alsh_1`

№	Входные данные	Выходные данные
1	5 0 3 2 6 13 9 0 10 9 11 6 16 0 5 6 6 6 11 0 9 10 14 3 5 0	34 0 2 3 1 4 0
2	5 0 8 1 18 5 8 0 15 4 12 1 15 0 4 1 18 4 4 0 10	24 0 2 4 3 1 0

	5 12 1 10 0	
--	-------------	--

Выводы

В ходе работы были реализованы точный и приближённый алгоритмы решения задачи коммивояжёра. Функционал включает поиск оптимального маршрута методом Хелда–Карпа на основе динамического программирования, а также построение приближённого решения с использованием жадного алгоритма ближайшего соседа. Проведённые реализации демонстрируют различие между точными и эвристическими подходами: алгоритм Хелда–Карпа обеспечивает нахождение минимального маршрута ценой высокой вычислительной сложности $O(2^N * N^2)$, тогда как жадный алгоритм обладает более высокой производительностью $O(N^3)$, но не гарантирует оптимальность результата. Дополнительно был реализован механизм генерации и сохранения матриц весов для тестирования алгоритмов на различных входных данных.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Файл main.py

```
def tsp_dp(n, graph):
    memo = [[-1] * n for _ in range(1 << n)]
    parent = [[-1] * n for _ in range(1 << n)]

    def get_dist(u, v):
        d = graph[u][v]
        return d if d > 0 or u == v else float('inf')

    def dp(mask, u):
        if mask == (1 << n) - 1:
            dist = get_dist(u, 0)
            return dist

        if memo[mask][u] != -1:
            return memo[mask][u]

        res = float('inf')
        for v in range(n):
            if not (mask & (1 << v)):
                d_uv = get_dist(u, v)
                if d_uv != float('inf'):
                    new_dist = d_uv + dp(mask | (1 << v), v)
                    if new_dist < res:
                        res = new_dist
                        parent[mask][u] = v

        memo[mask][u] = res
        return res

    min_cost = dp(1, 0)

    if min_cost == float('inf'):
        return "no path", None
```

```

path = []
curr_mask = 1
curr_city = 0
while curr_city != -1:
    path.append(curr_city)
    next_city = parent[curr_mask][curr_city]
    if next_city == -1: break
    curr_mask |= (1 << next_city)
    curr_city = next_city
path.append(0)

return min_cost, path

def als_1(n, graph):
    visited = [False] * n
    path = [0]
    visited[0] = True
    total_cost = 0
    curr = 0

    for _ in range(n - 1):
        next_city = -1
        min_d = float('inf')

        for v in range(n):
            if not visited[v] and graph[curr][v] > 0:
                if graph[curr][v] < min_d:
                    min_d = graph[curr][v]
                    next_city = v

        if next_city == -1:
            return "no path", None

        path.append(next_city)
        visited[next_city] = True
        total_cost += min_d
        curr = next_city

```

```

    if graph[curr][0] > 0:
        total_cost += graph[curr][0]
        path.append(0)
        return total_cost, path
    else:
        return "no path", None

def main():
    n = int(input())

    graph = []
    for _ in range(n):
        graph.append(list(map(int, input().split())))

    cost, path = tsp_dp(n, graph)

    if cost == "no path":
        print("no path")
    else:
        print(int(cost))
        print(*(path))

if __name__ == "__main__":
    main()

```